

# SafeSight: Driver Aid System for Blindspot Detection, Collision Warning, and Lane Departure.

Aryan Patel

May 30, 2026

## Abstract

Modern Advanced Driver Assistance Systems (ADAS) significantly reduce human-error-related traffic accidents; however, their reliance on proprietary, high-cost sensor arrays—such as LiDAR and millimeter-wave radar—restricts these critical safety features to premium vehicles. Thus, older vehicles and even some newer more economical vehicles do not get access to these safety enhancing features. To address this accessibility gap, this project introduces a low-cost, strictly vision-based ADAS framework engineered to democratize vehicle safety using affordable edge-computing hardware. The proposed system synthesizes lightweight deep learning (YOLOv8n) with classical geometric computer vision techniques, including Canny Edge Detection, Hough Transforms, and Exponential Moving Averages (EMA), to generate a robust perceptual safety buffer. Designed specifically for deployment on resource-constrained microcomputers like the Raspberry Pi 5, the architecture successfully executes spatial blindspot monitoring, monocular depth estimation for forward collision warnings, and dynamic lane departure tracking. Real-world testing validates the system’s efficacy, achieving a consistent 3–6 FPS processing rate alongside highly stabilized lane-tracking locks and accurate, LiDAR-free proximity alerts. Ultimately, this research demonstrates that life-saving predictive safety technologies can be viably retrofitted using economy hardware.

## 1 Introduction

Traffic accidents represent a critical global health crisis, with human error—such as driver distraction, delayed reaction times, and spatial inattentive blindness—accounting for the vast majority of motor vehicle collisions. While the advent of Advanced Driver Assistance Systems (ADAS) has proven highly effective at mitigating these risks, these life-saving technologies remain locked behind a significant financial paywall. Currently, millions of older, economy-class vehicles on the road lack even basic predictive safety features [1]. The primary motivation of this research is to democratize road safety by engineering a low-cost, retrofittable driver aid system capable of providing modern ADAS capabilities to older vehicles.

Modern factory-installed ADAS, such as those deployed by Tesla or Mercedes-Benz, rely heavily on complex sensor fusion architectures. These systems integrate data from proprietary, high-cost sensors, including LiDAR arrays, millimeter-wave (mmWave) radar [2]. Consequently, these solutions are burdened by severe financial and structural limitations; they cost thousands of dollars, demand heavy power consumption, and cannot be practically or economically integrated into a standard older vehicle, such as a 2010 sedan.

To bridge this accessibility gap, we introduce SafeSight: a purely vision-based perception pipeline that replaces expensive proprietary sensor hardware with optimized, smart software. The core concept of SafeSight is to utilize a standard, low-cost monocular camera feed processed entirely on accessible edge-computing hardware, specifically the Raspberry Pi 5. Because standard neural networks are too computationally expensive for localized micro-computers, SafeSight employs a hybrid algorithmic approach. By synthesizing lightweight deep learning (YOLOv8n) for high-level object classification with classical computer vision mathematics—such as Hough Transforms and Exponential Moving Averages (EMA)—the system maintains the processing speeds necessary for real-time hazard detection.

The primary contribution of this paper is demonstrating that a highly optimized, purely vision-based perception pipeline can execute life-saving ADAS logic on an readily available local computer, Raspberry Pi 5, without relying on cloud connectivity while also being fast enough to respond to threats accordingly.

- **Blindspot Detection:** A spatial geometric “No-Zone” mapping system that detects and warns the user if any car is in their blindspot.
- **Lane Departure Warning (LDW):** A robust lane-tracking system uniquely stabilized by an Exponential Moving Average (EMA) algorithm, effectively eliminating visual polygon flickering across sequential frames.
- **Rear Collision Warning:** A LiDAR-free, monocular depth-estimation tool that accurately calculates vehicle proximity using camera geometry.

Ultimately, this project culminates in a fully localized, functional system capable of running concurrent video safety feeds at a stable 3–6 Frames Per Second (FPS) on the Raspberry Pi 5, proving that advanced predictive safety can be effectively and affordably retrofitted.

## 2 Procedure

The core perception pipeline relies on deep learning frameworks and classical computer vision libraries. The object detection architecture utilizes the Ultralytics micro-framework to deploy the YOLOv8n (nano) model. This specific, lightweight iteration of the YOLO architecture was deliberately selected to optimize vehicle classification speeds on resource-constrained edge hardware.

All classical image manipulation and geometry generation were handled via the OpenCV (cv2) library, while the underlying matrix mathematics and spatial coordinate arrays were managed using NumPy. To extract physical lane boundaries for the Lane Departure Warning (LDW)

system, the pipeline utilized Canny Edge Detection coupled with Hough Line Transforms. Because classical line detection is highly susceptible to frame-to-frame visual flickering, an Exponential Moving Average (EMA) mathematical filter was applied to the coordinate arrays to ensure lane stabilization. Furthermore, the rear collision warning subsystem utilized standard monocular camera geometry to extrapolate depth estimations from two-dimensional bounding boxes. Experimental Design and Execution

Prior to feature extraction for blindspot detection and collision warning, all incoming video feeds underwent a pre-processing stage. This included applying fisheye undistortion matrices—configured with a balance of 0.1 and a scale of 0.7—to correct lens warping. Additionally, a lower-third of the camera was cropped was applied to eliminate interior vehicle dashboard interfering with detections, and creating a Region of Interest (ROI) on the roadway. Lane departure did not contain as much pre-processing as the other two systems as the dataset did not contain distorted images from fisheye cameras; it did contain the step for creating a region of interest similar to the other two systems.

Following pre-processing, the isolated logic pipelines were executed as follows: - Blindspot Detection: Hazard alerts were evaluated using point-polygon spatial intersection testing, determining if the bottom-center of the bounding box of a detected vehicle breached predefined geometric “No-Zone” arrays mapped to the side-mirror fields of view.

- Lane Departure Warning (LDW): Drift was calculated via dynamic pixel deviation. The algorithm continuously compared the absolute center-screen X-coordinate against the dynamically tracked center of the EMA-stabilized lane polygon.
- Rear Collision Warning: Distance scaling was computed by comparing the real-time pixel width of the YOLOv8n bounding boxes against an assumed standard physical vehicle width of 2.0 meters.

Overall system viability was quantified by measuring localized processing latency (Frames Per Second) and evaluating polygon tracking stability.

## Resources and Replication

All software components, source code, and model weights have been open-sourced to allow independent researchers to fully replicate this methodology. Furthermore, the complete suite of Python execution scripts, data processing Jupyter Notebooks are hosted on GitHub and can be cloned from Github Repo link [3] . The datasets can be found here lane Departure Data[4], Blindspot detection and collision warning [5].

## 2.1 Software and Environment

| Component     | Detail           |
|---------------|------------------|
| Language      | Python 3.11.9    |
| ML Framework  | PyTorch 2.3      |
| Key Libraries | OpenCV and NumPy |
| Hardware      | Raspberry pi 5   |

## 2.2 Data

The data utilized for system evaluation consisted entirely of two-dimensional image sequences capturing real-world driving environments. Because the perception pipeline leveraged the pre-trained YOLOv8n architecture for zero-shot vehicle classification, localized model training was not required. Consequently, while the dataset was initially structured with standard training and testing partitions, the data was exclusively utilized as a continuous testing and validation benchmark to measure system latency and algorithmic stability.

## 2.3 Algorithm / Approach

### 1. Spatial Blindspot Detection

The blindspot subsystem relies on geometric intersection testing. First, the raw camera frame undergoes fisheye undistortion to ensure linear accuracy. Next, the YOLOv8n model scans the frame to extract bounding box coordinates for all detected vehicles. Simultaneously, static “No-Zone” polygons are drawn on the frame, representing the physical blindspots adjacent to the host vehicle. The algorithm calculates the bottom-center centroid of each detected vehicle’s bounding box (representing the vehicle’s footprint on the road). If this centroid mathematically intersects with the No-Zone polygon, a blindspot alert is triggered.

### 2. Rear Collision Warning

The collision warning subsystem utilizes monocular depth estimation. The images were undistorted, YOLOv8n extracts vehicle bounding boxes in the forward path. Because the system lacks a physical depth sensor (like LiDAR), it estimates distance using pinhole camera geometry. The algorithm calculates the perceived pixel width of a detected vehicle and compares it against an assumed physical vehicle width (standardized to 2.0 meters) and the camera’s known focal length. If the calculated physical distance falls below a critical safety threshold, a collision warning is activated.

### 3. Lane Departure Warning (LDW) with EMA Stabilization

The lane tracking subsystem utilizes classical computer vision enhanced by mathematical filtering. After undistorted the image, a dynamic Region of Interest (ROI) mask is applied to isolate the lower third of the frame (the roadway). Canny Edge Detection and Hough Line Transforms extract the physical lane boundaries, which are then used to construct a lane polygon. The line detection inherently flickers between frames due to varying road conditions, an Exponential Moving Average (EMA) filter is applied. The EMA utilizes the mathematical coordinates of the previous frame’s lane lines to heavily influence the current frame’s lines, effectively smoothing the data and locking the polygon in place. Finally, the algorithm continuously tracks the absolute pixel center of this stabilized polygon against the static center of the camera frame; significant pixel deviation indicates a lane departure.

## 2.4 Experimental Design

The main metric that was measured was the speed that the systems ran on a raspberry pi, as the purpose of the project is to run detections on a system locally that would be attached to a car.

# 3 Results

## 3.1 Quantitative Results

The systems was able to run on a Raspberry Pi 5 at a consistent 3-6 fps. It was near the faster end for blindspot detection, which is expected as it mainly utilizes the YOLO model to detect cars and if they are in a certain region it give the user a warning. For the other two systems it ran slower as there was a lot more steps involved between preprocessing the images to get them ready for lane and car detection.

## 3.2 Figures and Visualizations

### Blindspot Detection



Figure 1: Fisheye Data Utilized for Blindspot Detection.



Figure 2: Data Undistorted with a Focus on the Regions Around the Car.

Detections for rgb\_03287\_RV.png

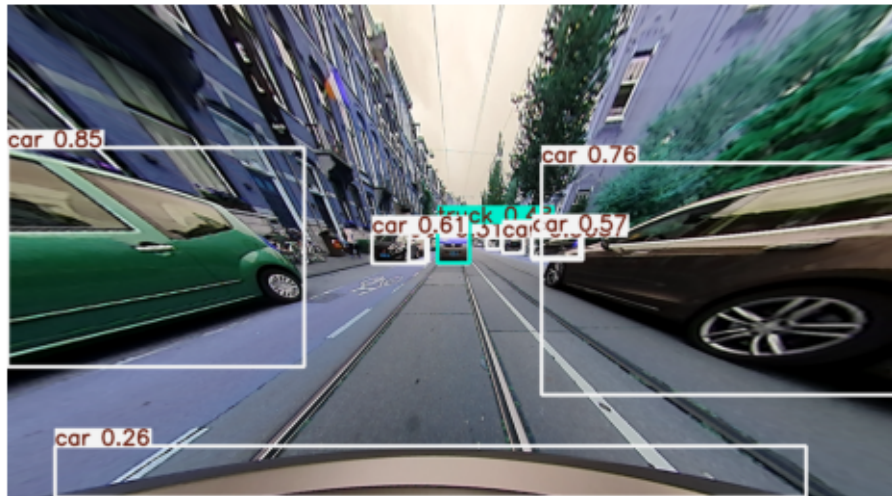


Figure 3: YOLOv8n Model Utilized on the Image.

Detections with Blind Spot Zones - rgb\_03287\_RV.png

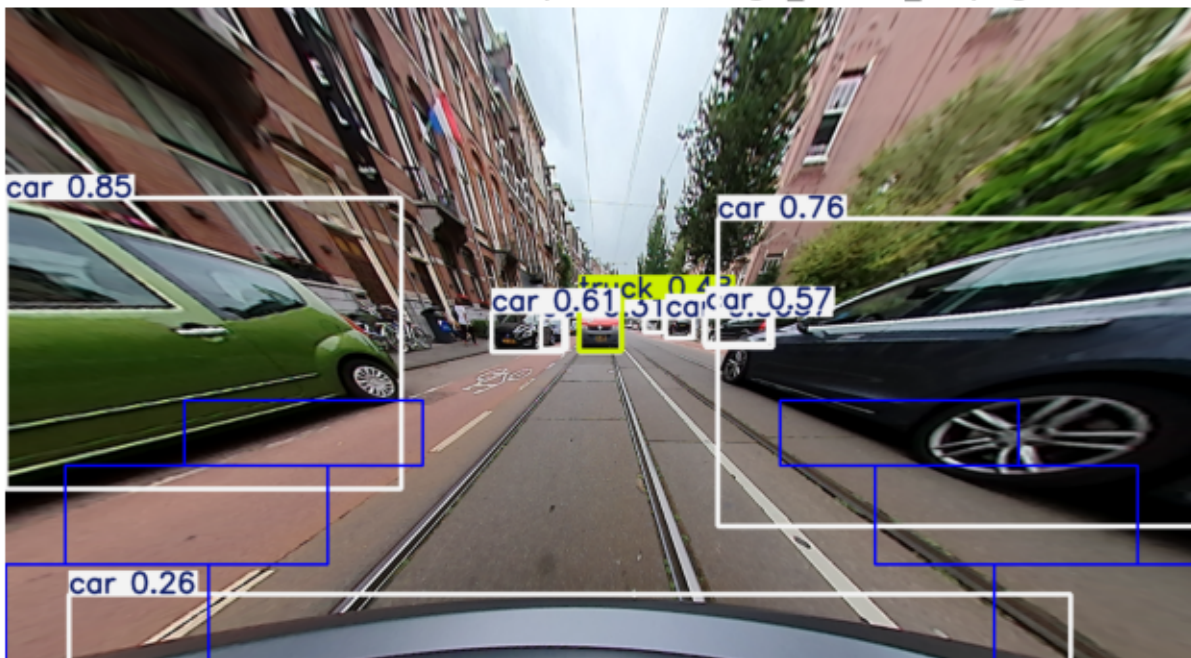


Figure 4: Blindspot Region Created.

## Lane Collision



Figure 5: Fisheye Example for Collision Warning.



Figure 6: Data Undistorted with a Focus on Depth.

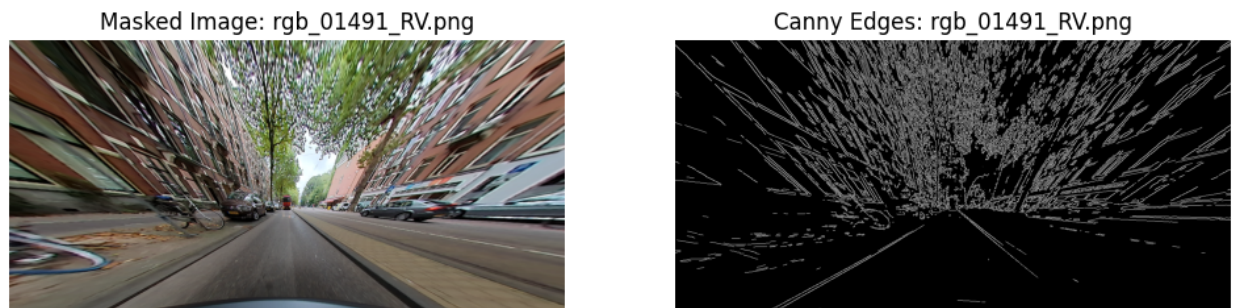


Figure 7: Canny Edge Detection.

ROI: rgb\_01491\_RV.png

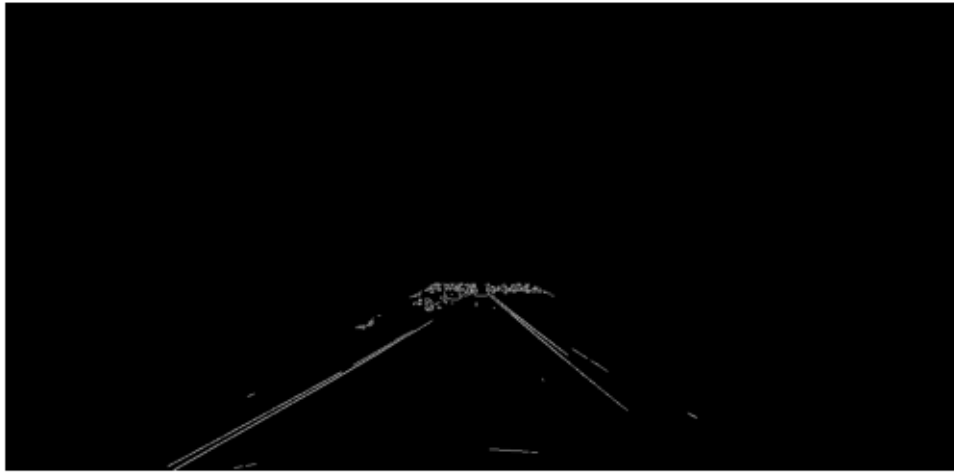


Figure 8: Created a Region of Interest.

Hough Lines: rgb\_01491\_RV.png

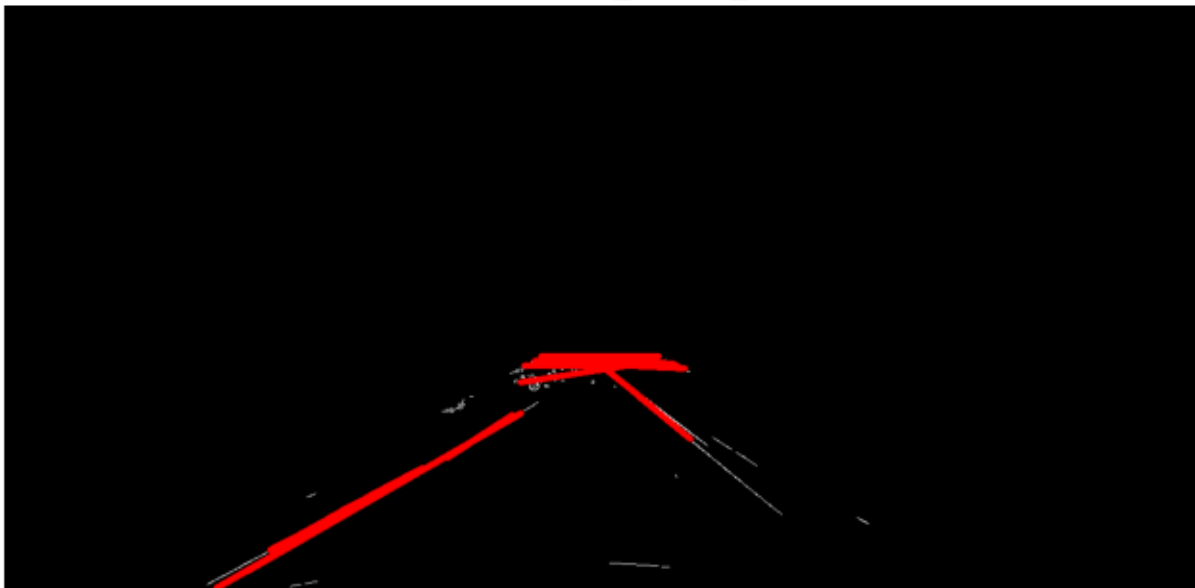


Figure 9: Hough Line Detection to Detect Possible Right and Left Lane Markings

Lane Zone & Rear Collision Check: rgb\_01491\_RV.png



Figure 10: Zone Created of the Lane.

**Lane Departure Warning** Lane departure warning contains the same steps as collision warning up until EMA, shown below:

Lane Zone & Rear Collision Check: ezgif-frame-089.jpg



Figure 11: Frame 89 without EMA.

Lane Zone & Rear Collision Check: ezgif-frame-090.jpg



Figure 12: Frame 90 without EMA.

Final ADAS Overlay: ezgif-frame-090.jpg



Figure 13: Frame 90 with EMA and Departure Warning.

## 4 Final Product

A website has been made that allows the user to test out the systems. By allowing the user to upload images and also to change any parameters they wish.

| Name                                      | Description                                                        |
|-------------------------------------------|--------------------------------------------------------------------|
| roi_top_ratio<br>number<br>(query)        | ROI top ratio (0–1). ~0.5 front, ~0.6 rear.<br>Default value : 0.5 |
|                                           | <input type="text" value="0.5"/><br>maximum: 1<br>minimum: 0       |
| canny_low<br>integer<br>(query)           | Canny lower threshold.<br>Default value : 50                       |
|                                           | <input type="text" value="50"/><br>minimum: 0                      |
| canny_high<br>integer<br>(query)          | Canny upper threshold.<br>Default value : 150                      |
|                                           | <input type="text" value="150"/><br>minimum: 0                     |
| departure_threshold<br>integer<br>(query) | Lane departure pixel threshold.<br>Default value : 40              |
|                                           | <input type="text" value="40"/><br>minimum: 0                      |

Request body **required**

multipart/form-data

image \* **required** Image frame to analyse (JPEG or PNG).  
string

Figure 14: Image of website

## 5 Conclusions

The SafeSight project successfully demonstrates that advanced, predictive vehicle safety systems do not inherently require prohibitive hardware investments. By supplanting traditional sensor fusion arrays—such as LiDAR and radar—with a purely vision-based software pipeline, this research establishes the viability of democratized Advanced Driver Assistance Systems (ADAS). Operating entirely on an accessible Raspberry Pi 5 microcomputer, the hybrid architecture successfully merged YOLOv8n deep learning with classical computer vision techniques to maintain a stable localized processing rate of 3–6 FPS. Furthermore, the integration of Exponential Moving Average (EMA) mathematical filters successfully mitigated visual flickering to provide highly stable lane tracking, while standard pinhole camera geometry proved highly effective for monocular depth estimation.

Despite these functional successes, the project’s reliance on localized edge computing introduced notable operational constraints. The primary hardware bottleneck of the Raspberry Pi 5 necessitated the deployment of the lightweight “nano” variant of the YOLOv8 architecture, which inherently compromises long-range detection accuracy compared to its larger

parameter counterparts. Additionally, relying exclusively on an optical perception pipeline introduces fundamental environmental vulnerabilities; purely monocular systems are highly susceptible to failure under adverse conditions such as heavy rain, fog, or direct solar glare. Retrospectively, while the individual subsystems were rigorously validated in isolated testing environments, a primary regret of this project scope is the absence of a unified, multi-camera physical vehicle rig, which would have facilitated synchronous, real-world testing of the fully integrated pipeline.

Looking forward, future iterations of this framework must prioritize hardware acceleration to achieve true real-time operational standards. Migrating the core perception pipeline to dedicated AI microcomputers, such as the NVIDIA Jetson or Google Coral TPU, would enable the system to easily surpass the 30 FPS real-time threshold. To address the inherent environmental vulnerabilities of optical lenses, future architectures should explore low-cost sensor fusion by integrating affordable millimeter-wave (mmWave) radar modules, ensuring necessary functional redundancy. Furthermore, this project could be transformed a little bit in order to work for motorcycles, as features such as rear end collision warning is really useful for motorists. This is because at red lights or stop signs a motorist is more likely to be rear ended as they are harder to see, especially if the driver is using their phone or preoccupied with something else. Thus, a small system to help them detect rear end collisions could prove to help the motorists to avoid collision or at least give them enough time to properly brace themselves for a collision.

## References

- [1] T. Neumann, “Analysis of advanced driver-assistance systems for safe and comfortable driving of motor vehicles,” *Sensors*, vol. 24, no. 19, 2024, doi: 10.3390/s24196223.
- [2] S. Ingle and M. Phute, “Tesla autopilot: Semi autonomous driving, an uptick for future autonomy,” *International Research Journal of Engineering and Technology*, vol. 3, no. 9, pp. 369–372, 2016.
- [3] AryanPJ, “SafeSight\_Senior.” [https://github.com/AryanPJ/SafeSight\\_Senior](https://github.com/AryanPJ/SafeSight_Senior), n.d.
- [4] rslim087a, “Lane Departure Data (road-video/test2.mp4).” <https://github.com/rslim087a/road-video/blob/master/test2.mp4>, n.d.
- [5] valeoai, “WoodScape: Blindspot detection and collision warning dataset.” <https://github.com/valeoai/WoodScape>, n.d.